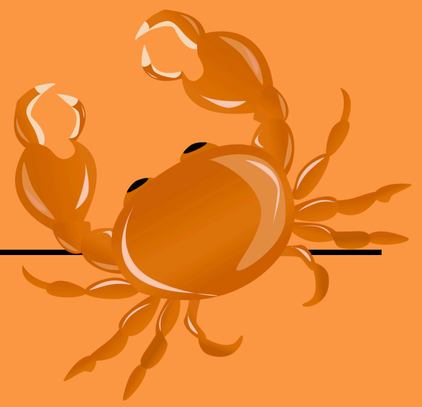
GRUPPO 14



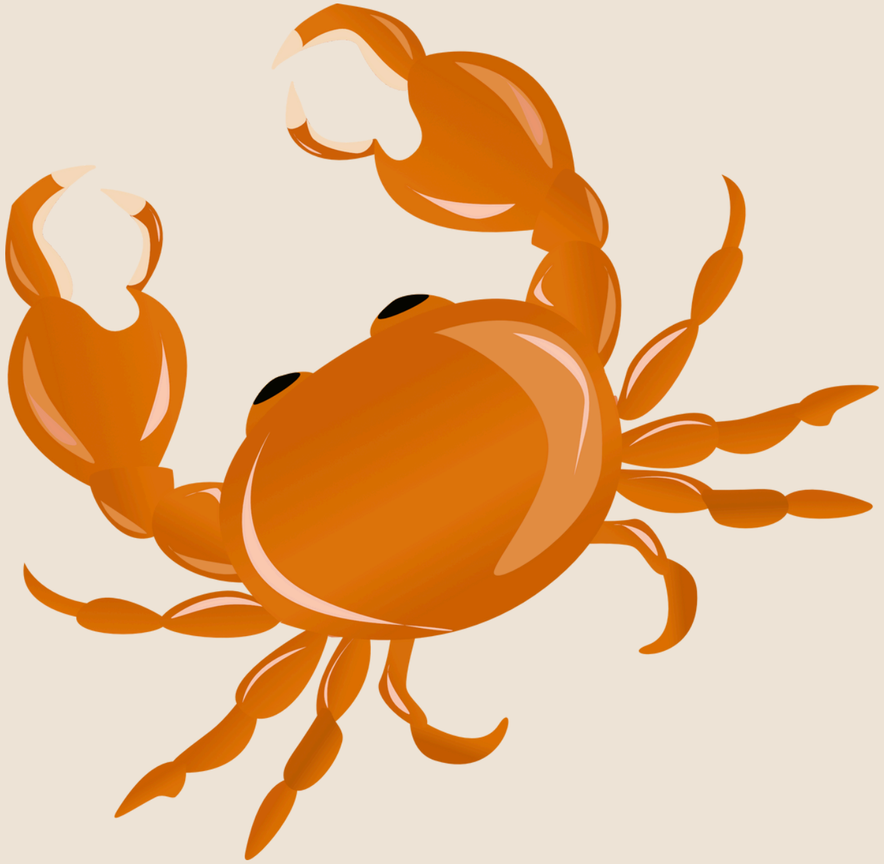
SISTEMA DI CHAT CLIENT-SERVER IN RUST

Comunicazione in tempo reale con logging delle performance

OBIETTIVI

Si vuole realizzare un sistema di chat di gruppo asincrono client-server per garantire comunicazione testuale in tempo reale

Si vuole analizzare la performance della CPU



SERVER CENTRALE

Il **server** gestisce la creazione di gruppi di chat, l'invio di inviti ad altri utenti e il instradamento dei messaggi verso i membri appropriati.

CLIENT MULTIPLI

I **client** sono applicazioni desktop: principalmente un'app con Interfaccia grafica (GUI) realizzata con il framework Iced, tramite la quale gli utenti possono registrarsi (login), creare gruppi, invitare partecipanti e inviare messaggi.

La **comunicazione tra client e server** avviene su connessioni TCP asincrone fornite da Tokio (tokio::net::TcpStream), usando un semplice protocollo testuale: ogni messaggio applicativo è serializzato in JSON e terminato da un carattere newline \n

DETTAGLI TECNICI

1

Tokio

Runtime asincrono per networking e task concorrenti

2

serde_json

Serializzazione JSON

3

Iced

Interfaccia grafica desktop

SERVER: CARATTERISTICHE

Il server lavora in maniera **asincrona**.

Si mette in ascolto su 0.0.0.0:8080 con un **TcpListener.**

Con **tokio spawn** avviamo una task che ogni 2 minuti logga la CPU.

```rust
let listener = TcpListener::bind("0.0.0.0:8080").await?;

tokio::spawn(async {
    let mut interval = time::interval(Duration::from_secs(120));
    loop {
        interval.tick().await;
        if let Err(e) = log_cpu_usage().await {
            eprintln!("Errore logging CPU: {}", e);
        }
    }
});
```

SERVER: CARATTERISTICHE

Entra in un ciclo infinito in cui aspetta **connessioni TCP** in ingresso (listener.accept()). Per ogni nuovo client crea un socket dedicato (stream.into_split()). stream.into_split() restituisce un OwnedReadHalf ed un OwnedWriteHalf.

Gruppi, utenti e inviti sono gestiti con **HashMap** protette da Arc<Mutex<>>. Infatti nel loop i puntatori ai dati globali (Arc<Mutex<>>) vengono clonati così che ogni client potrà accedere a gruppi, utenti e inviti in sicurezza.

Nel loop viene avviato un nuovo task Tokio con tokio::spawn per gestire quel client in parallelo. Questo è un task asincrono leggero, gestito dal runtime Tokio, che permette di servire molti client contemporaneamente.

```rust
let groups: Arc<Mutex<HashMap<String, Vec<Arc<Mutex<OwnedWriteHalf>>>>>> =
    Arc::new(Mutex::new(HashMap::new()));
let users: Arc<Mutex<HashMap<String, Arc<Mutex<OwnedWriteHalf>>>>> =
    Arc::new(Mutex::new(HashMap::new()));
let invites: Arc<Mutex<HashMap<String, HashSet<String>>>> =
    Arc::new(Mutex::new(HashMap::new()));

loop {
    let (stream, addr) = listener.accept().await?;
    let (reader, writer) = stream.into_split();
    let writer = Arc::new(Mutex::new(writer));
    let groups = Arc::clone(&groups);
    let users = Arc::clone(&users);
    let invites = Arc::clone(&invites);
    tokio::spawn(async move {
        if let Err(e) = handle_client(reader, writer, addr, groups, users, invites).await {
            eprintln!("Errore con client {}: {}", addr, e);
        }
    });
}
```

SERVER: CARATTERISTICHE

Entra in un ciclo infinito in cui aspetta **connessioni TCP** in ingresso (listener.accept()). Per ogni nuovo client crea un socket dedicato (stream.into_split()). stream.into_split() restituisce un OwnedReadHalf ed un OwnedWriteHalf.

Gruppi, utenti e inviti sono gestiti con **HashMap** protette da Arc<Mutex<>>. Infatti nel loop i puntatori ai dati globali (Arc<Mutex<>>) vengono clonati così che ogni client potrà accedere a gruppi, utenti e inviti in sicurezza.

Nel loop viene avviato un nuovo task Tokio con tokio::spawn per gestire quel client in parallelo. Questo è un task asincrono leggero, gestito dal runtime Tokio, che permette di servire molti client contemporaneamente.

```rust
let groups: Arc<Mutex<HashMap<String, Vec<Arc<Mutex<OwnedWriteHalf>>>>>> =
    Arc::new(Mutex::new(HashMap::new()));
let users: Arc<Mutex<HashMap<String, Arc<Mutex<OwnedWriteHalf>>>>> =
    Arc::new(Mutex::new(HashMap::new()));
let invites: Arc<Mutex<HashMap<String, HashSet<String>>>> =
    Arc::new(Mutex::new(HashMap::new()));

loop {
    let (stream, addr) = listener.accept().await?;
    let (reader, writer) = stream.into_split();
    let writer = Arc::new(Mutex::new(writer));
    let groups = Arc::clone(&groups);
    let users = Arc::clone(&users);
    let invites = Arc::clone(&invites);
    tokio::spawn(async move {
        if let Err(e) = handle_client(reader, writer, addr, groups, users, invites).await {
            eprintln!("Errore con client {}: {}", addr, e);
        }
    });
}
```

SERVER: CARATTERISTICHE

La funzione **handle_client** viene eseguita in un task Tokio dedicato per ogni client.
Entra in un loop infinito in cui:

- legge righe di testo dal socket TCP (con BufReader::read_line), attendendo JSON terminati da \n.
- Usa serde_json per deserializzare il testo in un Message.

Message è un enum condiviso da client e server, definito nella cartella common.

```rust
pub enum Message {
    Login { username: String },
    /// Un utente si unisce a un gruppo
    Join { username: String, group: String },
    /// Invita un utente in un gruppo
    Invite { group: String, user: String },
    /// Testo inviato in un gruppo
    Text { group: String, from: String, content: String },
    /// Acknowledgement generico
    Ack,
    /// Errore con motivazione
    Error { reason: String },
    Create { group: String },
    Leave  { group: String },
}
```

SERVER: CARATTERISTICHE

Inoltre **handle_client** gestisce il Message tramite pattern matching, eseguendo azioni diverse a seconda della variante:

- Login → registra l'utente e invia help + inviti
- Create → crea gruppo, aggiunge il creator come membro
- Join → controlla inviti e permette di entrare
- Leave → rimuove dal gruppo e notifica gli altri
- Text → inoltra messaggio ai membri del gruppo
- Invite → aggiunge invito e invia notifica se l'utente è online

Le risposte vengono serializzate in JSON e inviate al socket del client con write_all.

CLIENT: CARATTERISTICHE

Il client del progetto Ruggine è un’applicazione desktop sviluppata in Rust che permette agli utenti di connettersi al server di chat, creare gruppi, invitare altri utenti e scambiare messaggi in tempo reale.

È stato implementato principalmente in due versioni:

- Client Gui
- Client testuale

CLIENT GUI

Realizzato con il framework Iced, che adotta un'architettura Model-View-Update ispirata a Elm.

Permette all'utente di effettuare login, visualizzare gruppi, unirsi o lasciarli, invitare altri utenti e partecipare a chat di gruppo in modo interattivo.

Riceve i messaggi in arrivo dal server tramite un canale asincrono e aggiorna l'interfaccia in tempo reale.

CLIENT TESTUALE (CLI)

Applicazione a linea di comando, sviluppata per testare rapidamente la comunicazione col server.

Permette di inviare comandi come /login, /create, /join, /invite, /msg, visualizzando le risposte del server direttamente nel terminale.

Utilizzato durante la fase di debug e sviluppo.

GUI CLIENT: CARATTERISTICHE

- **Interfaccia grafica reattiva**: mostra l'elenco dei gruppi a cui l'utente ha accesso o è stato invitato, le chat attive e i popup per la creazione dei gruppi o l'invito di altri utenti.

- **Gestione asincrona della comunicazione**: usa Tokio per mantenere una connessione TCP aperta col server, leggere e inviare messaggi JSON senza bloccare l'interfaccia.

- **Notifiche in tempo reale**: riceve inviti, messaggi o errori e li mostra direttamente nella GUI, aggiornando il modello applicativo.

GUI CLIENT: ARCHITETTURA GENERALE

main.rs/

- **Model:** stato applicativo
- **View:** genera l'interfaccia in base allo stato
- **Update:** reagisce a eventi (input, click, messaggi server) aggiornando il model

network.rs/

- **connection :** funzioni principali di interazione tra server e gui_client

GUI_CLIENT: *network.rs*

All'interno del file stabiliamo la connessione e definiamo le due funzioni principali:

connect_and_listen(addr: &str):

- Apre la connessione TCP
- Imposta il writer globale
- Avvia una task che legge i messaggi json e li manda con tx.send alla gui

send_message(msg:&Message):

- Serializza un message in json e lo invia al server usando il Writer in modo asincrono

```rust
static WRITER: OnceCell<Arc<Mutex<tokio::net::tcp::OwnedWriteHalf>>> = OnceCell::new();

ALICE NIOSI s339225
pub async fn connect_and_listen(addr: &str) -> Result<UnboundedReceiver<Message>, Box<dyn std::error::Error>>
    let stream = TcpStream::connect(addr).await?;
    let (reader, writer) = stream.into_split();

    let writer_arc :Arc<?> = Arc::new(Mutex::new(writer));
    WRITER.set(writer_arc.clone()).map_err(|_| "WRITER già inizializzato")?;

    let (tx, rx) = unbounded_channel();

    tokio::spawn(async move {
        let mut buf_reader = BufReader::new(reader);
        let mut line :String = String::new();
        while buf_reader.read_line(&mut line).await.unwrap_or(0) > 0 {
            let trimmed = line.trim_end();
            if let Ok(msg) = serde_json::from_str::<Message>(trimmed) {
                let _ = tx.send(msg);
            }
            line.clear();
        }
    });

    Ok(rx)
}
```

```rust
pub async fn send_message(msg: &Message) {
    let json = serde_json::to_string(msg).unwrap() + "\n";
    if let Some(writer) = WRITER.get() {
        let mut w = writer.lock().await;
        let _ = w.write_all(json.as_bytes()).await;
        println!("🚀 Inviato linea JSON: {}", json.trim());
    } else {
        eprintln!("❌ WRITER non inizializzato!");
    }
}
```

GUI_CLIENT: *main.rs*

Il model tiene lo stato dell'applicazione con la **struct ChatGUI**

```
struct ChatGUI {
    username: String,
    login_input: String,
    current_group: Option<String>,
    groups: HashSet<String>,
    joined_groups: HashSet<String>,
    messages: HashMap<String, Vec<String>>,
    chat_input: String,
    group_input: String,
    invite_input: String,
    creating_group: bool,
    inviting_user: bool,
    rx: Option<Arc<Mutex<UnboundedReceiver<Message>>>>,
}
```

GUI_CLIENT: *main.rs*

Implementazione del trait Application di Iced per la struct ChatGUI
impl Application for ChatGUI indica a Iced come deve gestire:

- la creazione dell'app (con il metodo new)
- l'aggiornamento dello stato (con il metodo update)
- il rendering della GUI (con il metodo view)
- la gestione di input esterni (con il metodo subscription)

GUI_CLIENT: *main.rs*

`**new**` → inizializza il Model con tutti i campi vuoti o default

`**update**` → reagisce ai click, input e messaggi server aggiornando lo stato

`**view**` → genera la GUI a partire dallo stato

`**subscription**`→ ascolta messaggi asincroni da Tokio e crea eventi(cioè messagi messagegui che verranno poi processati da update)

`**title**` e `**theme**` → configurano la finestra

GUI_CLIENT:
Esempio principale di utilizzo

1 Login iniziale: view

Se username è vuoto, view mostra input e pulsante:

```rust
fn view(&self) -> Element<'_, MessageGUI> {
    if self.username.is_empty() {
        return Column::new()
            .padding(20)
            .spacing(10)
            .push(Text::new("Username:"))
            .push(TextInput::new("Username", &self.login_input).on_input(MessageGUI::LoginInput))
            .push(Button::new(Text::new("Login")).on_press(MessageGUI::LoginSubmit))
            .into();
    }
```

Premendo “Login” viene inviato l’evento MessageGUI::LoginSubmit.

GUI_CLIENT:
Esempio principale di utilizzo

2

Gestione in update

Il metodo update riceve MessageGUI::LoginSubmit e:

- salva username
- apre la connessione TCP con connect_and_listen
- manda Message::Login al server
- salva il receiver rx con MessageGUI::ReceivedRx

GUI_CLIENT:
Esempio principale di utilizzo

2

in fn update

```
MessageGUI::LoginSubmit => {
    let name = self.login_input.trim().to_string();
    if !name.is_empty() {
        self.username = name.clone();
        return Command::perform(async move {
            let rx = connect_and_listen("127.0.0.1:8080").await.expect("connessione fallita");
            send_message(&Message::Login { username: name }).await;
            Arc::new(Mutex::new(rx))
        }, MessageGUI::ReceivedRx);
    }
    Command::none()
}
```

GUI_CLIENT:
Esempio principale di utilizzo

3

Salva receiver Tokio

Sempre in update, quando arriva MessageGUI::ReceivedRx(rx)

```
MessageGUI::ReceivedRx(rx :Arc<?> ) => { self.rx = Some(rx); Command::none() }
```

Ora la GUI ha il canale su cui riceverà i messaggi dal server.

GUI_CLIENT:
Esempio principale di utilizzo

4

Un altro utente invia un messaggio:

- Il server inoltra il messaggio. Il task Tokio nella connect_and_listen lo riceve:

```
tokio::spawn(async move {
    let mut buf_reader = BufReader::new(reader);
    let mut line :String = String::new();
    while buf_reader.read_line(&mut line).await.unwrap_or(0) > 0 {
        let trimmed = line.trim_end();
        if let Ok(msg) = serde_json::from_str::<Message>(trimmed) {
            let _ = tx.send(msg);
        }
        line.clear();
    }
});
```

Il messaggio JSON viene deserializzato e mandato su tx.send(msg).

GUI_CLIENT:
Esempio principale di utilizzo

5 Subscription ascolta i messaggi: La GUI è in ascolto grazie a:

```rust
fn subscription(&self) -> Subscription<MessageGUI> {
    if let Some(rx :&Arc<?> ) = &self.rx {
        let rx :&Arc<?> = rx.clone();
        subscription::unfold("network", rx, |rx| async move {
            let msg = {
                let mut guard = rx.lock().await;
                guard.recv().await
            };
            if let Some(msg) = msg {
                (MessageGUI::Received(msg), rx)
            } else {                    (MessageGUI::ChatInput(String::new()), rx)
            }
        })
    } else {
        Subscription::none()
    }
}
```

Quando riceve un messaggio, genera MessageGUI::Received(msg).

GUI_CLIENT:
Esempio principale di utilizzo

6 fn update aggiorna lo stato

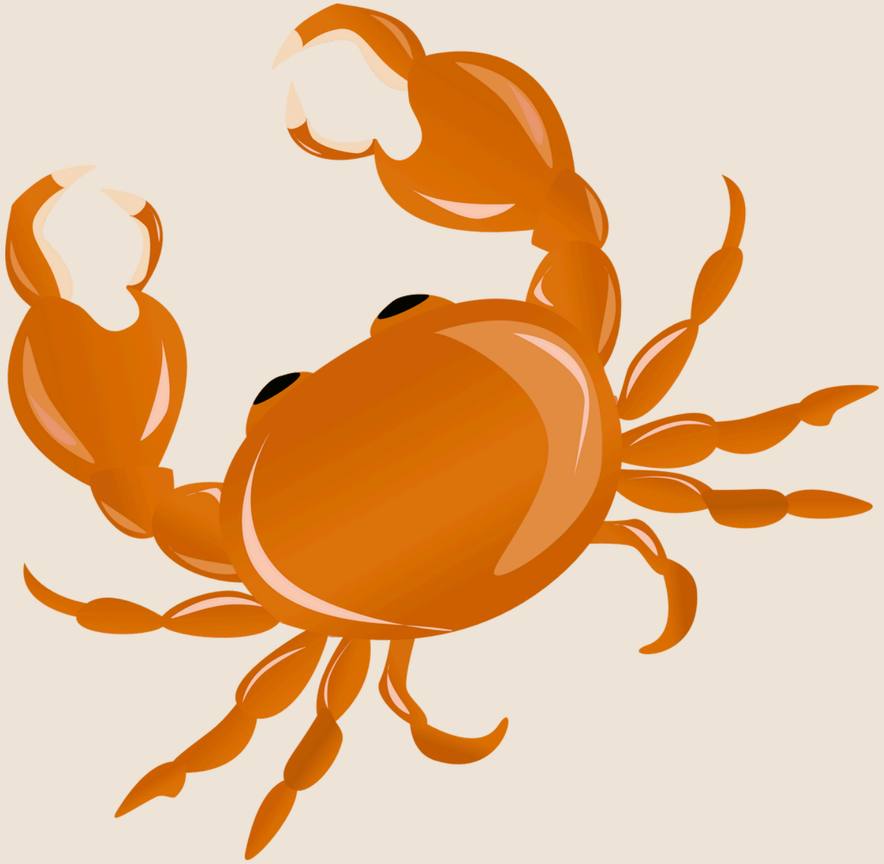
```
MessageGUI::Received(msg) => {
    match msg {
        Message::Text { group, from, content } => {
            if group != "system" {
                self.groups.insert(group.clone());
                self.messages : HashMap<...>
                    .entry(group.clone()) : Entry<String, Vec<...>>
                    .or_default() : &mut Vec<String, Global>
                    .push( value: format!("<{}>: {}", from, content));
            }
        }
```

Aggiunge il messaggio nella chat del gruppo.

GUI_CLIENT:
Esempio principale di utilizzo

7 View si aggiorna

- Al prossimo view, la GUI:
- legge self.messages aggiornato
- mostra i messaggi nella chat.

GRUPPO 14

GRAZIE PER L’ATTENZIONE!